GCD of Two Numbers Using Euclidean Algorithm

C Program

The following program finds the GCD of two unsigned n-bit numbers using the iterative division method.

```c
#include <stdio.h>
#include <time.h>

unsigned int gcd(unsigned int a, unsigned int b)
{
    unsigned int r;

    while(b != 0)
    {
        r = a % b;
        a = b;
        b = r;
    }

    return a;
}

int main()
{
    unsigned int a, b, result;
    clock_t start, end;
    long iterations = 1000000;

    printf("Enter two numbers:\n ");
    scanf("%u %u", &a, &b);

     // Timing code using clock() (developed with assistance
          from Claude Sonnet 5)
    start = clock();
    for (long i = 0; i < iterations; i++)
    {
        result = gcd(a, b);
    }
    end = clock();

    double total_time = (double)(end - start) /
        CLOCKS_PER_SEC;

    printf("gcd of %u and %u is %u\n", a, b, result);
    printf("Total clock cycles for %ld runs: %ld\n",
        iterations, (long)(end - start));
    printf("Total time: %f seconds\n", total_time);
```

```
    printf("Average time per call: %e seconds\n", total_time
        / iterations);

    return 0;
}
```

This program uses the Euclidean algorithm. At each step, the remainder $r = a \bmod b$ is computed, and then the values are shifted: a takes the old value of b, and b takes the value of r. This repeats until b becomes zero, at which point a holds the GCD.

For example, consider $\gcd(48, 18)$:

- $48 \bmod 18 = 12 \Rightarrow (a, b) \to (18, 12)$
- $18 \bmod 12 = 6 \Rightarrow (a, b) \to (12, 6)$
- $12 \bmod 6 = 0 \Rightarrow (a, b) \to (6, 0)$

Since $b = 0$, the loop exits and the program returns 6, which is indeed the GCD of 48 and 18.

Time Complexity Analysis

Let the input numbers be n-bit numbers.

The Euclidean algorithm follows the remainder sequence:

$$r_0 = q_1 r_1 + r_2$$

$$r_1 = q_2 r_2 + r_3$$

Substituting the second equation into the first:

$$r_0 = q_1(q_2 r_2 + r_3) + r_2$$

$$r_0 = (q_1 q_2 + 1) r_2 + q_1 r_3$$

Since:

$$q_1 q_2 + 1 \geq 2$$

we get:

$$r_0 > 2 r_2$$

This shows that after every two division steps, the remainder is reduced by at least a factor of 2.

For an n-bit input:

$$r_0 < 2^n$$

Since the remainder more than halves every two steps and must eventually fall below 1, it takes at most $2n$ such steps to reach that point. Therefore, the maximum number of division steps is:

$$k \leq 2n$$

Hence the time complexity is:

$$\boxed{T(n) = O(n)}$$

Exact Number of Steps

Let the while loop execute k times.

Initial operations:

$$\text{Initial steps} = 3$$

(Function call, declaration of r, and first loop condition check)

Each iteration performs:

$$r = a\%b$$

$$a = b$$

$$b = r$$

and one loop condition check.

Therefore:

$$\text{Steps per iteration} = 4$$

For k iterations:

$$4k$$

After the loop:

$$\text{Final steps} = 2$$

(Return statement and final loop condition check)

Therefore, total steps:

$$C(k) = 3 + 4k + 2$$

$$\boxed{C(k) = 4k + 5}$$

Since:

$$k \leq 2n$$

the maximum number of steps is:

$$C(n) = 4(2n) + 5$$

$$\boxed{C(n) = 8n + 5}$$

Clock Cycle Measurement

To measure actual runtime, `clock()` from `time.h` was used to time the `gcd()` function. The timing code was developed with assistance from Claude. Timing a single call directly gave a result of 0 clock cycles. This happens because `clock()` on most systems only has a resolution of a few milliseconds, while a single call to `gcd()` on 32-bit inputs finishes in well under a microsecond – both the start and end timestamps fall within the same tick.

To get a measurable result, the function was instead run in a loop of 1,000,000 iterations, and the total time was divided by the number of iterations to obtain the average time per call. This confirms that the function runs extremely fast and consistently for fixed-size (32-bit) inputs, which agrees with the $O(n)$ complexity derived above, since n is bounded by the word size.

Addition of Two n-bit Numbers Using Full Adders

To add two n-bit binary numbers, n full adders are connected in a chain, forming what is called a ripple carry adder. Each full adder takes three inputs: one bit from each of the two numbers, and a carry-in from the previous stage. It produces two outputs: a sum bit and a carry-out.

The addition begins at the least significant bit (LSB), where the carry-in is fixed at 0 since there is no previous stage. Moving toward the most significant bit (MSB), the carry-out of each full adder feeds directly into the carry-in of the next one – this is where the "ripple" in the name comes from.

Example

Consider the addition of the 4-bit numbers

$$0101_2 \ (5)$$

and

$$0011_2 \ (3).$$

The addition is performed from the least significant bit (LSB) toward the most significant bit (MSB), propagating the carry to the next stage.

$$
\begin{array}{cccccc}
 & C_3 & C_2 & C_1 & C_0 \\
 & 1 & 1 & 1 & 0 \\
\hline
 & 0 & 1 & 0 & 1 \\
+ & 0 & 0 & 1 & 1 \\
\hline
 & 1 & 0 & 0 & 0
\end{array}
$$

where

$$
\begin{aligned}
1 + 1 + 0 &= 0, \text{ carry } 1, \\
0 + 1 + 1 &= 0, \text{ carry } 1, \\
1 + 0 + 1 &= 0, \text{ carry } 1, \\
0 + 0 + 1 &= 1, \text{ carry } 0.
\end{aligned}
$$

Thus, the final result is

$$0101_2 + 0011_2 = 1000_2,$$

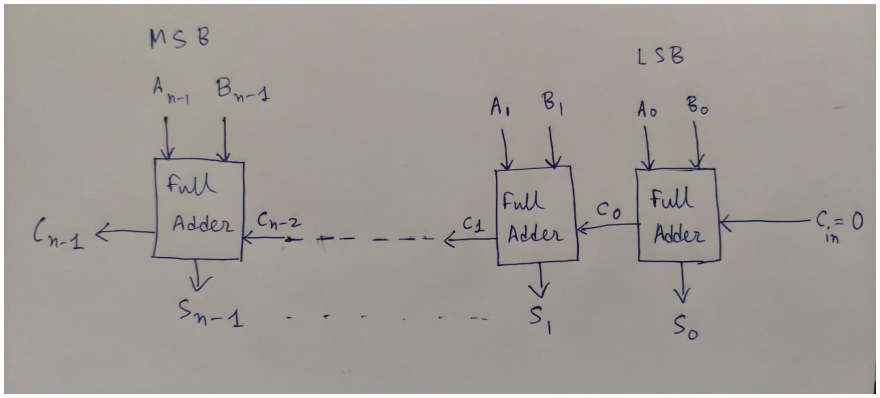
which is equivalent to

$$5 + 3 = 8.$$

For an n-bit addition, the final result has n sum bits plus one extra carry-out bit. If this final carry is 1, it signals overflow – the result needed $n + 1$ bits to represent correctly.

Full Adder Diagram

The diagram of the n-bit ripple carry adder is shown below.



References

1. Claude, Anthropic.
   `https://claude.ai`
   Used for assistance with the timing code implementation.